Spring Web MVC — XML 設定完整學習文件

> **適合程度**：有 Java 基礎、初次接觸 Spring MVC 的學習者 **學習目標**：能夠建立 Spring MVC 專案，理解 DispatcherServlet 流程，並使用常見的 Controller 注解

目錄

1. Spring MVC 核心概念
2. 請求流程圖解
3. 專案建立步驟
4. 設定檔詳解
5. Controller 注解大全
6. 常見錯誤與陷阱
7. 練習題

---

1. Spring MVC 核心概念

什麼是 Spring MVC?

Spring MVC（**Model-View-Controller**）是 Spring Framework 提供的 Web 框架，用於建構 Java Web 應用程式。

角色	英文	負責什麼
模型	Model	存放資料與業務邏輯
視圖	View	呈現畫面（JSP、HTML）
控制器	Controller	接收請求、處理邏輯、回傳結果

核心元件：DispatcherServlet

DispatcherServlet（前端控制器 Front Controller）是整個 Spring MVC 的入口點。

- 所有 HTTP 請求都先通過 DispatcherServlet
- 它讀取設定檔，將請求分派給對應的 Controller
- 取得 Controller 回傳的 View 名稱後，交由 ViewResolver 渲染畫面

---

2. 請求流程圖解

```
瀏覽器發出請求
      │
      ▼
 DispatcherServlet   ← 讀取 web.xml 設定
      │
      ▼
```

```
  HandlerMapping      ← 找到對應的 @Controller 方法
       │
       ▼
   Controller         ← 執行業務邏輯，回傳 View 名稱
       │
       ▼
  ViewResolver        ← 在 /WEB-INF/views/ 找到 .jsp 檔
       │
       ▼
    View (JSP)        ← 渲染 HTML 回應給瀏覽器
```

XML 設定檔命名規則

DispatcherServlet 預設讀取設定檔的規則：

```
/WEB-INF/{servlet-name}-servlet.xml
```

範例：若 `web.xml` 中 servlet-name 為 `myspringconfig`，則自動讀取：

```
/WEB-INF/myspringconfig-servlet.xml
```

> 若要自訂設定檔名稱，可在 `web.xml` 中使用 `contextConfigLocation` 參數覆寫。

---

3. 專案建立步驟

Step 1 — 建立 Maven Web 專案

在 Eclipse / IntelliJ 中建立 **Maven Web Project**，確認專案結構如下：

```
src/
  main/
    java/
      demo/springmvc/
        HelloWorldController.java
    webapp/
      WEB-INF/
        views/
          helloworld.jsp
        myspringconfig-servlet.xml
        web.xml
pom.xml
```

Step 2 — 設定 pom.xml 依賴

```xml
<dependencies>
    <!-- Spring MVC 核心 -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
        <version>5.2.11.RELEASE</version>
    </dependency>

    <!-- JSP 標準標籤庫 -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>jstl</artifactId>
        <version>1.2</version>
    </dependency>

    <!-- 單元測試（可選） -->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.11</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

> **重點**：`spring-webmvc` 已包含 `spring-core`、`spring-beans`、`spring-context` 等核心模組，無需重複加入。

---

4. 設定檔詳解

4.1 web.xml — 入口設定

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
    version="4.0">

    <display-name>HelloWorldSpring</display-name>

    <!-- 1. 宣告 DispatcherServlet -->
    <servlet>
        <servlet-name>myspringconfig</servlet-name>
        <servlet-class>
            org.springframework.web.servlet.DispatcherServlet
        </servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
```

```
    <!-- 2. 攔截所有請求交給 DispatcherServlet 處理 -->
    <servlet-mapping>
        <servlet-name>myspringconfig</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>
```

標籤	說明
`<servlet-name>`	決定自動讀取哪個 `-servlet.xml` 設定檔
`<load-on-startup>1</load-on-startup>`	應用程式啟動時立即初始化，不等到第一次請求
`<url-pattern>/</url-pattern>`	攔截所有 URL（靜態資源也會被攔截，需額外設定）

4.2 myspringconfig-servlet.xml — Spring MVC 設定

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans-4.1.xsd
      http://www.springframework.org/schema/context
      http://www.springframework.org/schema/context/spring-context-4.1.xsd
      http://www.springframework.org/schema/mvc
      http://www.springframework.org/schema/mvc/spring-mvc-4.1.xsd">



    <!-- 1. 掃描此 package 內有 @Controller 等注解的類別 -->
    <context:component-scan base-package="demo.springmvc"/>

    <!-- 2. 啟用注解驅動（支援 @RequestMapping 等）  -->
    <mvc:annotation-driven/>

    <!-- 3. 設定 View 解析器：前綴 + 後綴 = 實際 JSP 路徑 -->
    <bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/views/"/>
        <property name="suffix" value=".jsp"/>
    </bean>

</beans>
```

ViewResolver 路徑組合範例：

```
Controller 回傳值 → "helloworld"
prefix           → "/WEB-INF/views/"
suffix           → ".jsp"
實際路徑          → /WEB-INF/views/helloworld.jsp
```

---

5. Controller 注解大全

5.1 基本 Controller — `@Controller` + `@RequestMapping`

HelloWorldController.java

```java
package demo.springmvc;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller                             // 標記此類為 Spring MVC 控制器
public class HelloWorldController {

    @RequestMapping("/hello")           // 處理 GET /hello 的請求
    public String hello(Model model) {
        // 將資料放入 Model，JSP 可用 ${greeting} 取得
        model.addAttribute("greeting", "Hello Spring MVC");
        return "helloworld";            // 回傳 View 名稱（不含路徑與副檔名）
    }
}
```

helloworld.jsp

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <title>Spring MVC - Hello World</title>
</head>
<body>
    <h1>${greeting}</h1>   <%-- 顯示 Model 中的 greeting 值 --%>
</body>
</html>
```

5.2 重新導向 — `redirect:`

當方法回傳值加上 `redirect:` 前綴，Spring 會執行 **HTTP 302 重新導向**，而非渲染 View。

```java
@Controller
public class RedirectExampleController {

    @RequestMapping(value = "/redirect", method = RequestMethod.GET)
    public String doRedirect(Model model) {
        return "redirect:/hello";   // ← 重新導向到 /hello
    }

    @RequestMapping("/hello")
    public String hello(Model model) {
        model.addAttribute("greeting", "Hello Spring MVC");
        return "helloworld";
    }
}
```

redirect vs forward 比較:

	`redirect:`	`forward:`
HTTP 狀態碼	302	無額外請求
瀏覽器 URL	**會改變**	不變
用途	跨 URL 跳轉	內部轉發（共用 Model）

5.3 查詢參數 — `@RequestParam`

從 URL 的 `?key=value` 格式取得參數。

```java
@Controller
public class RequestParamExampleController {

    // 對應 URL: /user?name=Alice  或  /user (name 預設為 "Guest")
    @RequestMapping("/user")
    public String userInfo(
            Model model,
            @RequestParam(value = "name", defaultValue = "Guest") String name) {

        model.addAttribute("name", name);

        if ("admin".equals(name)) {
            model.addAttribute("email", "admin@demo.com");
        } else {
            model.addAttribute("email", "Not set");
        }
        return "userInfo";
    }
}
```

常用參數說明：

屬性	說明	範例
`value`	URL 參數名稱	`"name"`
`defaultValue`	若參數不存在時的預設值	`"Guest"`
`required`	是否必填（預設 true）	`required = false`

userInfo.jsp

```
<body>
    <h2>${name}</h2>
    Email: ${email}
</body>
```

5.4 路徑變數 — `@PathVariable`

從 URL 路徑本身提取變數，適合 RESTful 風格的 API。

```
@Controller
public class PathVariableExampleController {

    // 對應 URL: /web/fe/default/en/document/8080/spring-mvc
    @RequestMapping("/web/fe/{sitePrefix}/{language}/document/{id}/{naturalText}")
    public String documentView(
            Model model,
            @PathVariable("sitePrefix") String sitePrefix,
            @PathVariable("language") String language,
            @PathVariable("id") Long id,
            @PathVariable("naturalText") String naturalText) {

        model.addAttribute("sitePrefix", sitePrefix);
        model.addAttribute("language", language);
        model.addAttribute("id", id);
        model.addAttribute("naturalText", naturalText);

        String documentName = (id == 8080) ? "Spring MVC for Beginners" : "Java
Tutorial";
        model.addAttribute("documentName", documentName);

        return "documentView";
    }
}
```

URL 匹配範例：

```
/web/fe/default/en/document/8080/spring-mvc
              ↑       ↑          ↑      ↑
         sitePrefix language    id  naturalText
```

documentView.jsp

```
<body>
    <h3>${documentName}</h3>
    Site Prefix: ${sitePrefix} <br>
    Language: ${language}      <br>
    ID: ${id}                  <br>
    Natural Text: ${naturalText}
</body>
```

5.5 直接寫回 HTTP 回應 — `@ResponseBody`

加上 `@ResponseBody` 後，回傳值**直接寫入 HTTP 回應 Body**，不再透過 ViewResolver 解析 View。

```java
@Controller
public class ResponseBodyExample1Controller {

    @RequestMapping("/saveResult")
    @ResponseBody                       // ← 直接輸出字串，不找 JSP
    public String saveResult() {
        return "saved";                 // 瀏覽器收到純文字 "saved"
    }
}
```

> **進階補充**：在現代 Spring Boot 開發中，通常直接用 `@RestController`（等同於 `@Controller` + `@ResponseBody`）來建構 REST API，回傳 JSON 物件。

---

6. 常見錯誤與陷阱

錯誤情境	原因	解法
`404 Not Found`	`@RequestMapping` 路徑打錯，或 component-scan 沒掃到	確認 URL 路徑與 `base-package` 設定
JSP 找不到（白畫面或錯誤）	prefix/suffix 設定錯誤，JSP 放錯目錄	確認 ViewResolver 路徑組合正確
靜態資源（CSS/JS）404	`url-pattern=/` 攔截了所有請求包含靜態資源	在 xml 中加入 `<mvc:default-servlet-handler/>`

錯誤情境	原因	解法
`@RequestParam` 報錯	必填參數不存在時拋出 `MissingServletRequestParameterException`	加上 `required = false` 或 `defaultValue`
redirect 後 Model 資料消失	redirect 是新的 HTTP 請求，Model 不跨請求保留	改用 `RedirectAttributes` 傳遞 Flash Attribute
XML 命名空間設定錯誤	`xsi:schemaLocation` 少填或版本錯誤	複製官方範本，確認 beans/context/mvc 命名空間都完整

---

7. 練習題

Easy — 建立第一個 Hello World 頁面

題目：建立一個 `/greet` 的路徑，Controller 將 `"Welcome to Spring!"` 放入 Model，並顯示在 JSP 上。

▶ 提示
▶ 解答

Medium — 使用 @RequestParam 製作計算機

題目：建立 `/calc?a=10&b=5` 路徑，將兩數相加的結果顯示在頁面上。

▶ 提示
▶ 解答

Hard — 使用 @PathVariable 設計商品查詢頁

題目：設計 `/shop/{category}/{productId}` 路徑，根據 `productId` 判斷商品名稱：

- `1001` → `"Java Book"`
- `1002` → `"Spring Book"`
- 其他 → `"Unknown Product"`

▶ 解答

學習筆記摘要

注解 / 設定	用途	關鍵字
`@Controller`	標記 Spring MVC 控制器	Bean 管理
`@RequestMapping`	對應 URL 路徑到方法	路由 Routing
`@RequestParam`	取得 URL 查詢參數 `?key=value`	查詢字串 Query String
`@PathVariable`	取得 URL 路徑中的變數 `/{id}`	RESTful URL
`@ResponseBody`	直接輸出到 HTTP 回應，不找 View	REST API

注解 / 設定	用途	關鍵字
`redirect:`	HTTP 302 重新導向到另一個 URL	重定向
`InternalResourceViewResolver`	根據前綴+後綴找到 JSP 檔案	View 解析
`DispatcherServlet`	Spring MVC 的核心前端控制器	Front Controller

> **現在試試看**：依照本文步驟，從建立 Maven 專案開始，完成 Hello World 並成功在瀏覽器顯示 `"Hello Spring MVC"`，再嘗試完成三道練習題！